IEEE SHA Tech Team

Developer Onboarding & Git Workflow Guide

Welcome to the team! We are building a scalable MERN stack platform. To ensure our architecture remains stable and collaborative, we operate with a strict version control workflow. The `master` branch is securely locked—this guide will walk you through exactly how to set up your environment and contribute code safely.

Phase 1: The One-Time Setup (Your First Day)

You only need to complete these steps once to configure your laptop for the project.

Step 1: Accept the Invitation & Clone the Repo

Accept the GitHub invitation sent to your email. Then, open your terminal and download the codebase to your machine:

```
git clone https://github.com/ieeesha10-svg/Ieee-website.git
cd Ieee-website
```

Step 2: The "Magic" Git Configuration

To prevent you from getting trapped in confusing terminal editors (like Vim) and to make syncing your code effortless, run these two commands right now:

```
# 1. Force Git to use VS Code for all messages
git config --global core.editor "code --wait"

# 2. Create the "Magic Sync" shortcut
git config --global alias.sync "!git checkout master && git pull
origin master && git checkout @{-1} && git merge master"
```

Step 3: Install Dependencies & Setup Environment

Our project is split into a frontend (React) and a backend (Node/Express). Install the packages for both:

```
cd client
npm install
cd ../server
npm install
```

> **Security Notice:** The backend requires a `.env` file to connect to the database. Reach out to the Tech Head to get the secret keys. Create the `.env` file inside the `server/` folder and paste the keys there. **Never upload this file to GitHub!**

Phase 2: The Daily Workflow (Every time you code)

Because the `master` branch is protected, you will follow this exact 4-step loop for every task you complete.

1. Claim a Task & Create a Branch

Go to the GitHub **Projects** board, find a task, and drag it to "In Progress". Then, in your terminal, create a new branch for your work. Never code on the master branch!

```
git checkout -b feat/your-name-task-name
```

2. Write Code & Commit

Write your code, test it locally, and save your progress with clear commit messages.

```
git add .
git commit -m "Developed the responsive student profile layout"
```

3. The Safety Sync (Prevent Conflicts)

Before you upload your code, you must ensure it doesn't conflict with updates made by other developers. Run the shortcut you created in Phase 1:

```
git sync
```

If there are conflicts, VS Code will highlight them. Choose the correct code, save the file, and commit the fix locally.

4. Push & Open a Pull Request (PR)

Upload your finished, synced branch to the GitHub server:

```
git push origin feat/your-name-task-name
```

Finally, go to the GitHub website. You will see a green banner at the top of the repository. Click **Compare & pull request**, write a brief description of what you did, and click **Create pull request**.

Phase 3: Code Review & Hand-off

Stop Coding! Once your Pull Request is open, your job for that specific task is done.

The Tech Head will review your code.

- If there are issues, you will receive comments requesting changes. You just fix them on your laptop, commit, and push again—the PR updates automatically.
- If the code is approved, the Tech Head will merge it into `master`.
- Once merged, you can delete your branch locally (`git branch -d feat/your-name-task-name`) and start the cycle over for your next task!